

Naqsha – Engineering Credibility Brief

Interview preparation document · compiled 2026-07-29 from the repository itself. Every number below is mined from the repo (git history, PR list, test suite, ADRs) – nothing is estimated.

1. The 30-second pitch

Naqsha is a design studio for pen plotters and laser cutters – a maker composes generative patterns as layers, routes modulation between them, attaches decorative motifs, assigns fabrication settings (cut / score / engrave / pen), and exports an SVG their machine faithfully reproduces. It's named for the *naqsheh*, the painted grid-sheet a Persian carpet designer hands to a weaver: the design lives between the designer and the machine, and the software's job is to make that handoff trustworthy.

The interview-relevant claim: this is a **single-owner product built end-to-end** – domain modeling, rendering engine, real-time editor UX, 3D material preview, database + auth + row-level security, export pipeline, and a written architectural decision record – with an AI-orchestrated workflow where I set direction, made the calls, and gated every merge.

2. Numbers at a glance (all mined from the repo)

Metric	Value	Source
Commits on <code>main</code>	865 (since 2026-03-26)	<code>git log</code>
Merged pull requests	67	<code>gh pr list -s merged</code>
Test files / tests	551 files · 7,621 tests passing (155s wall)	<code>vitest run</code>
Architecture Decision Records	8 (<code>docs/adr/</code>)	repo
Generative pattern engines	~24 (flow fields, phyllotaxis, Islamic star, space colonization, reaction-diffusion dash, Voronoi, moiré...)	<code>src/lib/patterns/</code>
Database migrations	15 (Supabase/Postgres)	<code>supabase/migrations/</code>
Domain glossary	35+ locked terms with explicit <i>avoid</i> lists	<code>CONTEXT.md</code>
Design brief	living document governing every UI decision	<code>.impeccable.md</code>

Four months, solo, part of the calendar time running as reviewed autonomous overnight builds (see §7).

3. Tech stack – and why each piece

Layer	Choice	The "why" worth saying out loud
UI	React 19 + Vite 8	SPA editor; no SSR framework because the product is a local-first canvas tool, not a content site. Vite for sub-second iteration on a large suite.
Generative engine	p5.js at build-time + custom generators	Patterns are pure functions (<code>params, seed</code>) → <code>geometry</code> . Crucially, modulation is applied at geometry build time, not draw time , so SVG export bypasses the canvas context entirely – the file the machine gets is computed from the same geometry the screen shows.
3D preview	three.js + React Three Fiber + drei/postprocessing	Physically-grounded material preview (see ADR-0003). Khronos PBR Neutral tone mapping instead of the ACES default, chosen deliberately for true-to-life material color.
Backend	Supabase (Postgres + Auth + RLS)	Solo-scale backend leverage: auth, row-level security policies, orgs/tiers/entitlements – with RLS policies tested live (<code>npm run test:rls</code> runs the suite against real policies, not mocks).
Fonts/text	opentype.js	Text layers resolve to actual vector outlines (plotters need paths, not <code><text></code>), including single-line Hershey engraving fonts. New fonts are validated through an opentype <code>getPath</code> gate before acceptance.
Image → vector	potrace (+ vendored jimp stub)	Photo extraction into plottable centreline/contour geometry; the jimp dependency is a local stub vendored to keep the bundle lean.
Testing	Vitest 4 + Testing Library + jsdom	7,621 tests. TDD was the default working mode for feature waves (red-green-refactor, adversarial review passes on top).
Styling	Tailwind CSS 3	Utility styling under a strict design system (<code>.impeccable.md</code>) that bans the generic dark-panel "creative tool" look.

Deliberate absences are part of the signal: no state-management library (document state is a single reducer-style store with global undo), no node-graph library (rejected twice, see ADR-0004), no TypeScript on this codebase – an honest tradeoff to name: velocity-first JS with a very large test suite carrying the correctness load instead. Own that answer rather than dodge it.

4. Architectural decisions that carry interview weight

These are written down as ADRs in `docs/adr/` – being able to hand an interviewer a decision record with rejected alternatives is itself the competence signal. The five most load-bearing:

4.1 Export never blocks; the receipt closes the safety hole (ADR-0001)

A UX review recommended gating export behind a machine-preview step. I **reversed the recommendation**: `⌘E` export stays one keystroke and never blocks, because most makers post-process in other tools (LightBurn, AxiDraw utility). The silent-failure risk is closed differently – every export runs the same `fabricationPipeline` preflight and emits a one-line **Export Receipt** (estimated run time, paths cropped, warning count). Because cropping runs *as a pipeline stage*, the plan, the receipt, and the file **agree by construction** – there is no second code path to drift. *Interview frame: judgment call against an expert review, with the underlying risk still resolved – plus "agree by construction" as a design principle.*

4.2 Exactly two reversibility geographies (ADR-0002)

Everything in the editor is live-edit + one global undo (the Figma/Ableton model). The **one** place a deliberate preview → apply → revert cycle exists is the Run Plan – the commit-to-machine boundary, where geometry

optimization (simplify/merge/reorder-for-travel) is only legible against the plan's travel/time numbers. Applied optimize values live *outside* the undo stack (their way back is the plan's own Revert) but persist with the document – previously they were bare `useState` and silently vanished on reload, changing what export produced. *Interview frame: a principled state-model boundary – "where does undo end and commit begin" – plus a real bug that the principle caught.*

4.3 Linear chain, not a node graph – twice (ADR-0004, ADR-0007)

Motif anchor selection is a reorderable **Chain of Blocks** (Ableton rack model): filters and a sequencer the anchor stream flows through, order being document state. A node graph (split/merge/patch cables) was explicitly rejected: it demands a scheduler, cycle detection, and a graph editor UI *before the first flower renders*, and multi-branch routing is already expressible by stacking motif layers. When the raster Etch feature later needed the same rack shape, I rejected generalizing Chain/Block across domains (ADR-0007) – an anchor-stream filter and a pixel-field transform share nothing but "ordered and reorderable," so the Etch Stack got its own vocabulary (Stages) rather than forcing one abstraction to mean two things. *Interview frame: resisting premature generality – the most senior-sounding thing in the whole repo. "The same shape is not the same abstraction."*

4.4 Determinism as a contract: hash-per-anchor RNG (ADR-0005)

Every render is reproducible from a seed. When chains became user-reorderable, sequential RNG streams became a liability (any upstream edit re-rolls every downstream anchor), so new randomized blocks derive values as `mulberry32(hash(seed, anchorId, channel))` – randomness as a pure function of the anchor, giving **edit locality**: anchors that survive an edit keep their random values. The legacy jitter stream was deliberately *not* migrated, because unifying it would visibly re-roll every existing document on upgrade. Two RNG idioms coexist, and the ADR says why. *Interview frame: backward compatibility as a first-class constraint; understanding why determinism structure matters (edit stability, not just replayability).*

4.5 WYSIWYG raster on mirror acrylic (ADR-0006) and fidelity-first 3D (ADR-0003)

The Etch layer breaks the codebase's own "everything is vector" rule on purpose: photos dither to a **locked 1-bit bitmap** embedded in the SVG, because any downstream re-dither (e.g. LightBurn re-screening a greyscale) is exactly the control loss the feature exists to eliminate – one stray dot scars mirror acrylic irreversibly. Vectorizing the dots was rejected with arithmetic: 10^4 – 10^5 dots at 254 DPI means per-dot travel (hours, not minutes). A terminal **Highlight Hold** clamp guarantees zero dots above a cutoff – applied *after* screening so no error-diffusion can violate it. Meanwhile the 3D preview renders marks as physical **reactions** (frost, char, kerf) under PBR-neutral tone mapping – when fidelity and legibility conflict, fidelity wins and the 2D canvas remains the legibility view. *Interview frame: physical-world constraints driving software architecture – the machine and the material are part of the system.*

Cross-cutting: a ubiquitous language, enforced

`CONTEXT.md` is a 35-term domain glossary where every entry has an *avoid* list ("Sheet, not canvas/work-area"; "Operation, not cut-setting"; "Block, not node – node implies a graph"). Features are named before they're built, ADRs amend the glossary, and reviews check code against it. This is domain-driven design practiced for real, at solo scale.

5. Big-picture feature map

The core loop: compose generative pattern layers → route modulation between them → adorn with motifs → assign operations → preview on material → run-plan → export.

- **Pattern layers** – ~24 parametric engines (flow fields, phyllotaxis, Islamic star/girih, Voronoi, space-colonization branching, moiré pairs, topographic contours...), each seeded and deterministic, plus imported SVG,

extracted-photo, and text layers.

- **Modulation** – one layer's field guides another's geometry (density, warp), applied at build time so exports match the screen. Flat routing edges, deliberately not a graph.
- **Motif system** (the deepest subsystem) – vector glyphs (58 built-ins + user library + a full pen-path editor) attach to *semantic hosts* on patterns: path tips, edges, junctions, cells. A reorderable **Chain** of Blocks filters the anchor stream; a terminal **Sequencer** deals glyph Slots in rhythm – optionally partitioned into structural **Zones** (Apex/Stem/Cell), which is how one vine layer flowers at path ends and leafs along the body (ADR-0008). Per-glyph overrides on top.
- **Fabrication model** – machine profiles (laser / plotter / drag cutter), an operation library (process + power/speed/passes or pen/pressure) where library order *is* machine execution order, pen-swap planning, sheet-vs-bed distinction, travel-time estimation.
- **Run Plan** – a shell-morph destination (not a modal, not a route): the same canvas re-renders as the machine's view – paths tinted by operation, travel dashed, crops ghosted – with the optimize stack and its preview→apply→revert cycle. Free for every tier as deliberate policy: operability is table stakes; design capability is what's paid.
- **3D material preview** – the piece on a table, unlit: calibrated material archetypes (clear/fluorescent acrylic, plywood...), marks as physical reactions.
- **Raster Etch** – photo → tone/dither/halftone **Etch Stack** → locked 1-bit engrave bitmap, with the Highlight-Hold floor for mirror stock.
- **Platform** – Supabase auth, orgs/memberships, tier entitlements, cloud save + local drafts, guest onboarding, all under live-tested RLS policies.

6. What makes it stand out from other design software

The honest competitive frame – know each neighbor and why Naqsha isn't it:

1. **vs. Illustrator / Inkscape:** those are general vector editors where fabrication is an afterthought (export and pray). Naqsha's *document model* is fabrication-native – operations, machine profiles, run time, pen swaps, and sheet cropping are first-class objects, and the export receipt means no silent failure. Generative patterns are parametric layers you re-roll, not frozen geometry.
2. **vs. LightBurn / machine software:** those own the *machine* side and do it well; Naqsha owns the *design* side and hands off cleanly (deliberately – ADR-0001 refuses to be a toll booth between the maker and their machine software).
3. **vs. creative-coding tools (Processing, vsketch, penplot):** the plotter-art community lives in code. Naqsha gives the same generative depth – seeds, modulation, parametric families – behind direct manipulation, layers, undo, and a library, with no code required.
4. **vs. node-graph tools (Grasshopper, TouchDesigner):** deliberately linear racks instead of graphs (ADR-0004) – the Ableton insight that a rack you reorder is more learnable than a graph you wire, and expressive enough when you can stack layers.
5. **The aesthetic stance is a product decision:** a written anti-reference list bans the generic dark-panel creative-tool skin; the rule is "*borrow the mechanic, refuse the chrome*" – Figma's inspector model, Adobe's tool strip, Ableton's racks, dressed in the naqsheh's paper-and-gouache. Light mode is the default because paper is the ground.
6. **Determinism as a promise:** same document, same seed, same bytes – across sessions and versions (ADR-0005 exists precisely to keep old documents byte-identical). For small-batch makers, reproducibility is the product.

7. How it was built – the AI-native workflow (own this, don't hide it)

The repo is co-built with AI agents, and the *orchestration is the achievement* – the honest framing is "I run a small AI engineering team and I own every decision," not "I typed all of this." The receipts live in the repo:

- **Direction documents steer the agents:** `.impeccable.md` (design constitution), `CONTEXT.md` (domain glossary), and 8 ADRs are the steering artifacts – agents build inside them, and drift gets caught against them.
- **Orchestrated feature waves:** `docs/*-ORCHESTRATOR.md` files script multi-agent builds (plan → TDD build → adversarial review → run report). Overnight autonomous runs shipped reviewed, test-covered waves – e.g. the Raster Etch feature landed as 8 sequential TDD waves, each adversarially reviewed.
- **Adversarial review as a gate:** PRDs get a written adversarial pass before build (e.g. `docs/PRD-143-adversarial-review.md`); reviews have overruled designs – the zoned-sequencer recursion strategy was switched from point-warp to corner-mean *because* an adversarial review caught the flaw.
- **Human gates are explicit:** a standing `NEEDS-HUMAN.md` ledger marks what autonomous runs must not decide (visual judgment calls, database migrations, anything irreversible). Decision documents record which calls were mine (e.g. export-never-blocks, rejecting the node graph, per-glyph angle semantics).
- **The test suite is the floor:** 7,621 tests run green as the merge baseline; TDD-first waves mean agent-written code arrives with its tests, and regressions surface immediately.

Interview frame: this answers the "how do you use AI" question with process, not tooling – direction documents, adversarial verification, explicit human gates, and a test floor. That's engineering management applied to AI agents.

8. Deploying this in the interview

If asked "tell me about a project": §1 pitch → the core loop (§5, one breath) → one deep story. Best default story: **the Chain-not-graph decision (§4.3)** – it has a rejected alternative, a reasoned cost model, and a sequel (refusing to generalize the abstraction later).

If asked a systems-design question: reach for **agree-by-construction (§4.1)** – one pipeline feeding plan, receipt, and file – and **the two reversibility geographies (§4.2)**.

If asked about testing/quality: live-RLS tests, the 7,621-test floor, TDD waves, adversarial review gates (§7).

If asked about tradeoffs/regrets: no-TypeScript (§3, own it), and the two coexisting RNG idioms (§4.4) – a deliberate inconsistency with a written justification beats a clean-looking migration that re-rolls every user's document.

If asked "what would you do differently": answer from the ADRs' *rejected alternatives* – you already did the analysis; that's the point of keeping them.

Likely hard follow-ups to have ready:

- "Why no TypeScript?" – velocity tradeoff, test suite carries correctness; global CLAUDE.md prefers strict TS for new work, and this is the named exception.
- "Solo project – how do I know you work on teams?" – the workflow **is** team practice: written decision records, review gates, a glossary that keeps naming consistent across contributors (human or agent).
- "Is it live? Who uses it?" – answer honestly with current status; do not inflate. [[ASK: current deployment/user status – fill in before the interview.]]

Sources: `package.json`, `docs/adr/0001-0008`, `CONTEXT.md`, `.impeccable.md`, `git log`, `gh pr list`, `vitest run` – all as of 2026-07-29 on branch `feat/motif-lines-updates` (metrics measured against `main`).